

Week 2 - Introduction to ROS2

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



WE ARE

HUMBER

Review from Last Week



Performance Objectives

1. Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions.
2. Configure ROS2 on a Linux-based system.
3. Develop ROS2 nodes in Python or C++ using rclpy or rclcpp.
4. Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices.
5. Utilize ROS2 topics, services and actions for inter-node communication.
6. Integrate hardware components (sensors and actuators) in a ROS2 system.
7. Implement coordinate frame management using the tf2 library to track and transform between reference frames in robotic applications.
8. Utilize simulation environments such as Gazebo with ROS2.
9. Investigate robot behavior using simulation environment such as Gazebo for testing and debugging.
10. Design a complete robotic application using ROS2 to solve open-ended engineering problems.

Review from Last Week



- **What is Linux**

- **Linux** is an open-source operating system kernel used to build powerful, secure, and stable operating systems.
- Popular Linux distributions (distros) include **Ubuntu, Debian, Fedora, and CentOS**.
- It is widely used in servers, embedded systems, cloud infrastructure, and robotics.

- **What is Python**

- **Overview:**
 - **Python** is a high-level, interpreted programming language known for its readability and simplicity.
 - Popular in fields like data science, web development, automation, and **robotics**.
- **Key Features:**
 - Easy to learn and read
 - Interpreted (no need to compile)
 - Vast library ecosystem
 - Used by major tech companies and research labs

Agenda for today's training session



Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions.
- Configure ROS2 on a Linux-based system

Problem we will be solving today:

- What is ROS?
- What is a development environment?
- Distributions, Nodes, Topics and Messages
- Services and actions
- Parameters and Remapping

What is ROS?



- **ROS (Robot Operating System) is an open-source framework for developing robot applications.**
- **Provides a collection of tools, libraries, and conventions to simplify building complex and robust robot behavior.**
- **Not an operating system itself, but a middleware that runs on top of Linux.**
- **Enables message-passing between processes ("nodes") in a distributed system.**

ROS2

Why ROS?



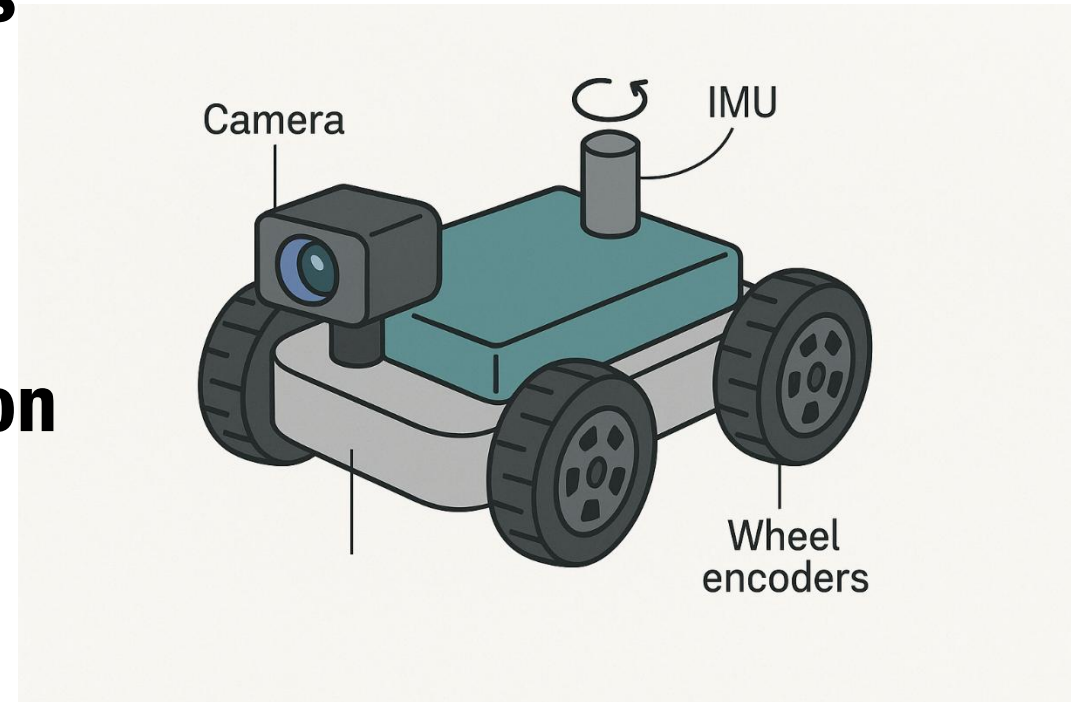
- **A framework for structuring robotics applications**
- **Provides tools for interacting with robots**
- **Offers pre-built packages for common functionality**
- **Makes it easier and faster to write robotics applications**

 **ROS2**

The Challenge of Complexity



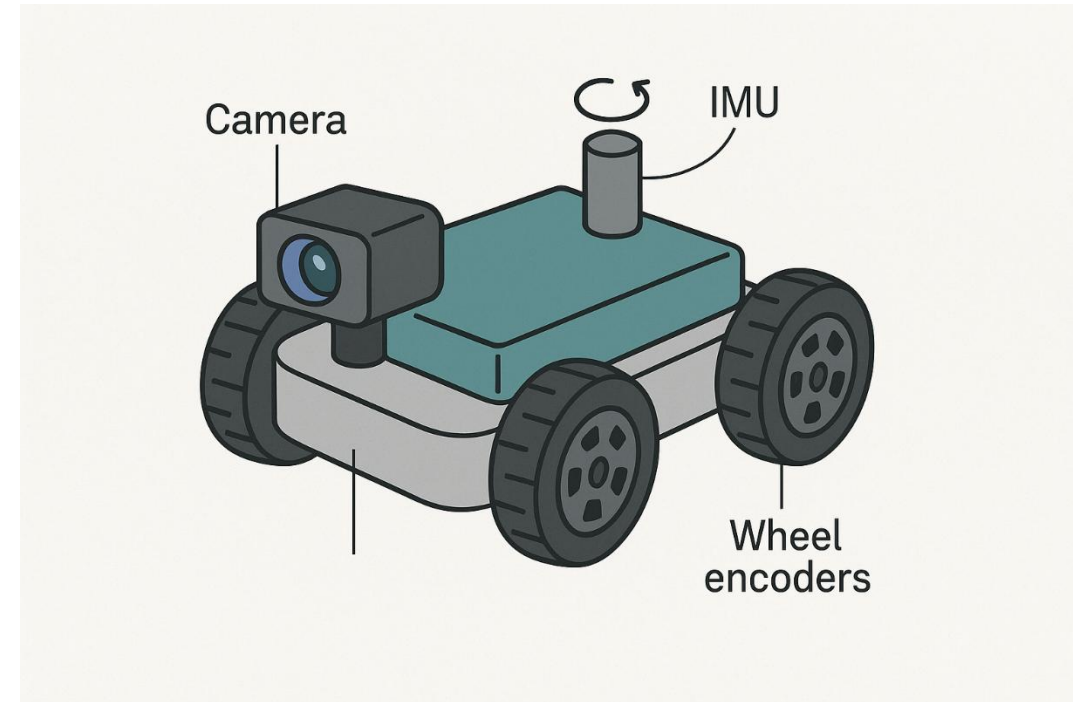
- **Complex robots need multiple sensors and systems working together**
- **Writing one huge program makes development harder**
- **Example: Mobile robot with localization using**
 - **Camera**
 - **IMU**
 - **Wheel/encoders**



ROS 2 Approach



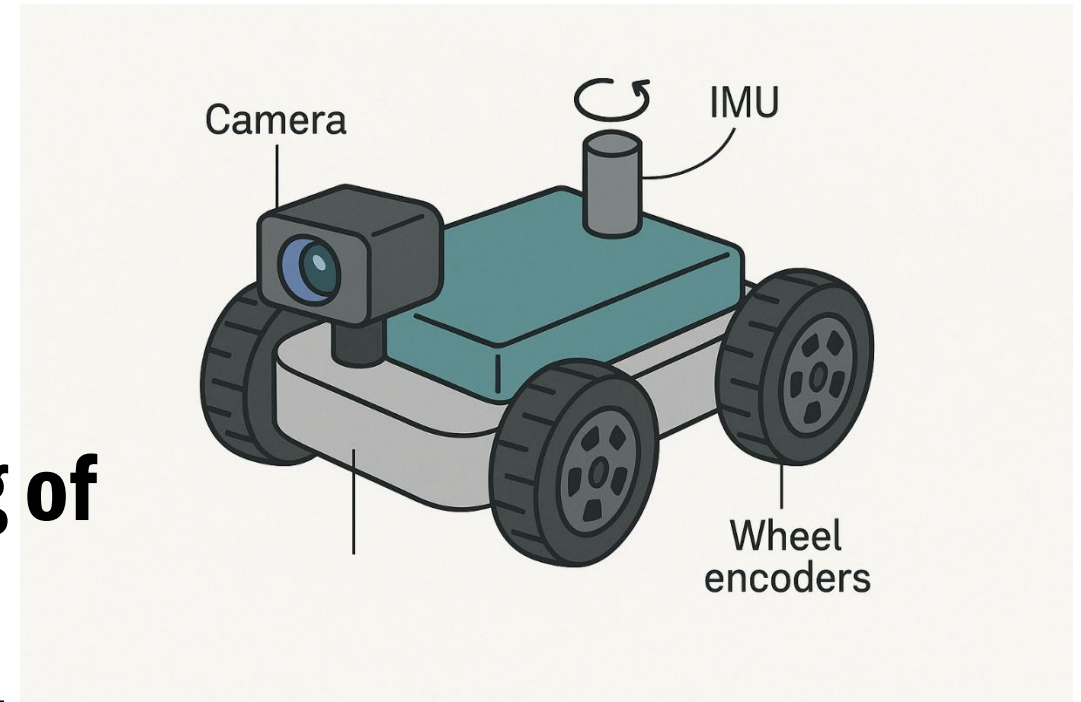
- **Breaks software into nodes with specific responsibilities**
- **Nodes communicate with each other through ROS 2 middleware**
- **Example:**
 - **Localization node combines data from camera, IMU, and wheel encoder nodes**



Benefits of This Structure



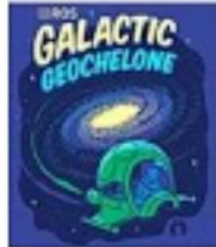
- **Easier to scale and add new functionality**
- **Supports distributed development across teams**
- **Enables inspection and debugging of system components**
- **Improves modularity and flexibility**



What is ROS?



WHAT IS ROS2



<https://www.youtube.com/watch?v=IFOtB9oPUo>

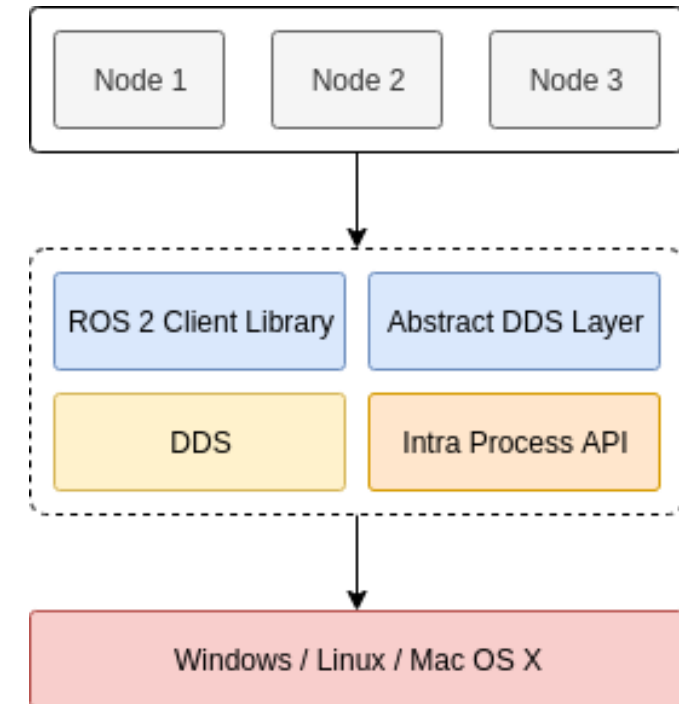
ROS2 Architecture



Distributed System: Composed of independent nodes communicating via topics, services, or actions.

Middleware: Built on DDS (Data Distribution Service), enabling real-time, scalable communication.

Modularity: Each component of a robot (e.g., perception, planning, actuation) can run as a separate node.



Q/A



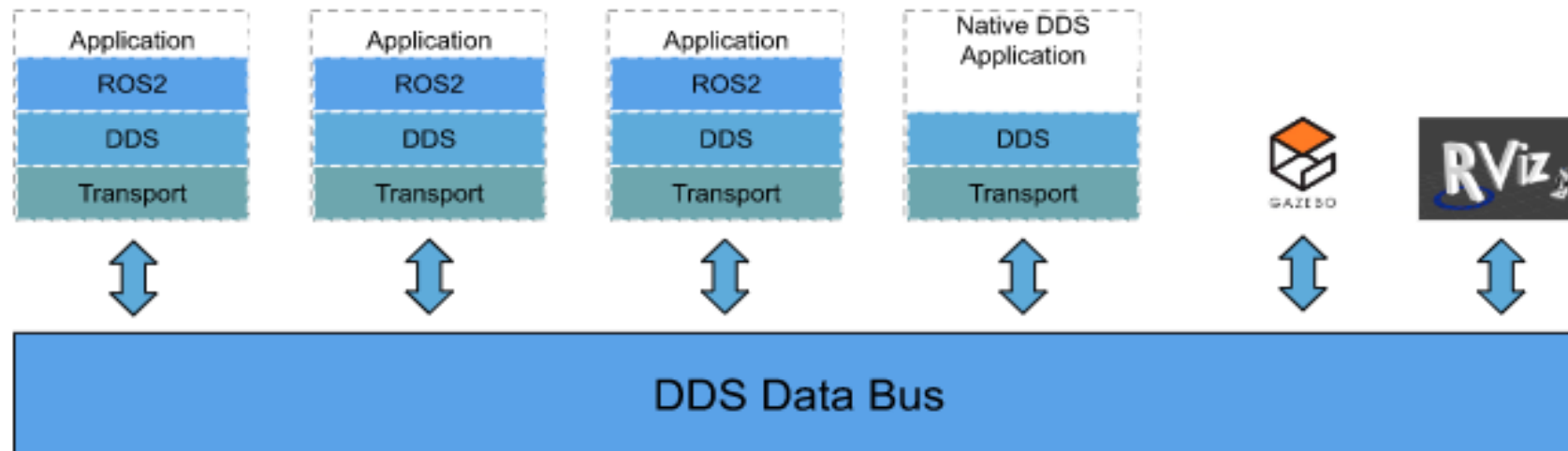
Which of the following are not components of ROS2 Architecture?

- a. Linux**
- b. Python**
- c. Nodes**
- d. Windows**

Answer: A,B & D

ROS2 Architecture- Data Distribution Service (DDS)

- **DDS is a publish-subscribe protocol standard used by ROS2.**
- **Facilitates real-time, secure, and scalable communication.**
- **Abstracts the network layer for ROS2 nodes.**
- **Enables ROS2 to support features like Quality of Service (QoS), which allows fine control over data delivery.**



ROS Distribution



- **A set of libraries and tools for building robotic systems**
- **New ROS distributions released frequently**
- **Designed to run on the latest Ubuntu LTS**
- **Best supported on Ubuntu (works on Linux/Mac/Windows, but less mature)**
- **Developed and maintained by Open Robotics**

 ROS2

ROS 2 Jazzy (Current LTS)



- **Latest LTS distribution: ROS 2 Jazzy**
- **Runs on Ubuntu 24.04 LTS**
- **Focuses on stability and long-term support for developers**
- **Provides updated libraries, middleware, and APIs**



Why Use Jazzy?



- **Long-term support = reliable for different projects**
- **Compatible with modern hardware and tools**
- **Actively maintained and improved by the ROS community**
- **Strong documentation and resources available**



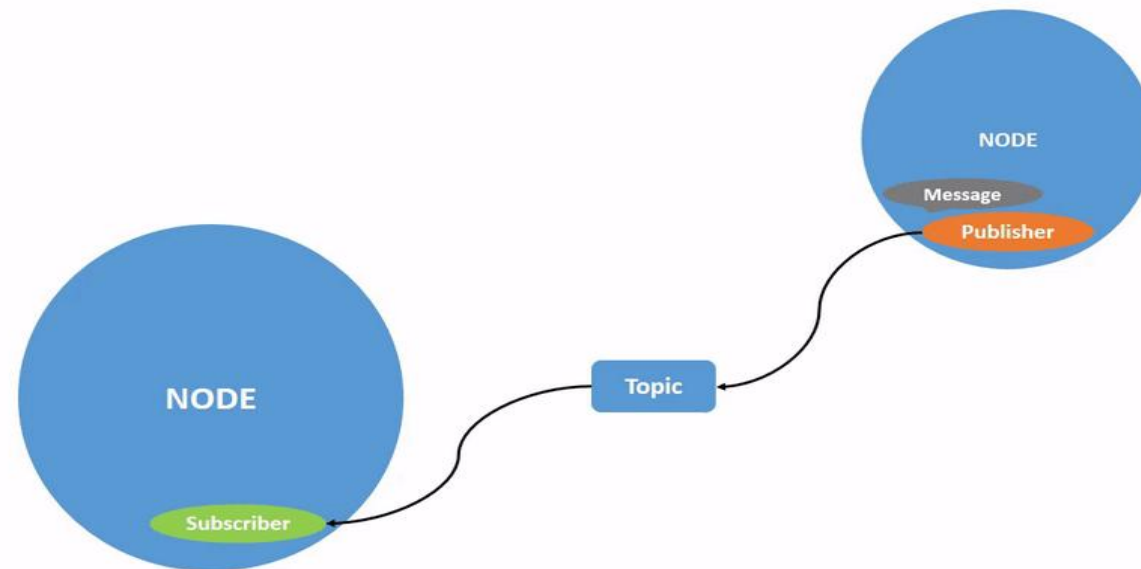
ROS 2 documentation (latest distributions, tutorials, installation guides):

docs.ros.org

Nodes



- **A ROS 2 node is a program that can use ROS libraries**
- **Nodes perform one specific task in the system**

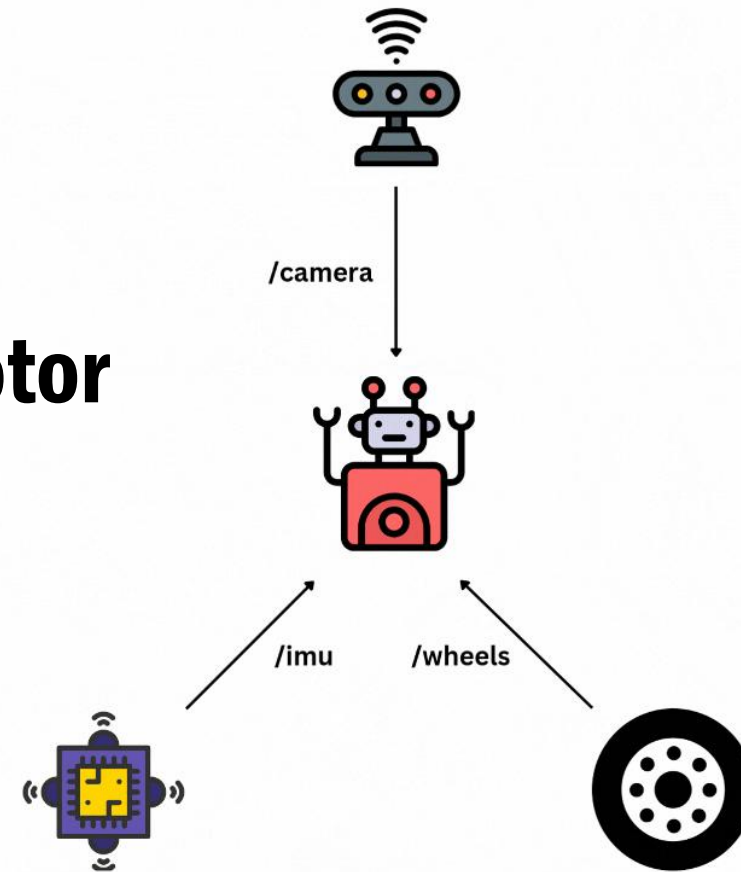


Nodes



Examples of node tasks:

- **Reading data from a sensor**
- **Sending control signals to a motor**
- **Receiving joystick input**
- **Displaying visualizations**
- **Computing a trajectory**

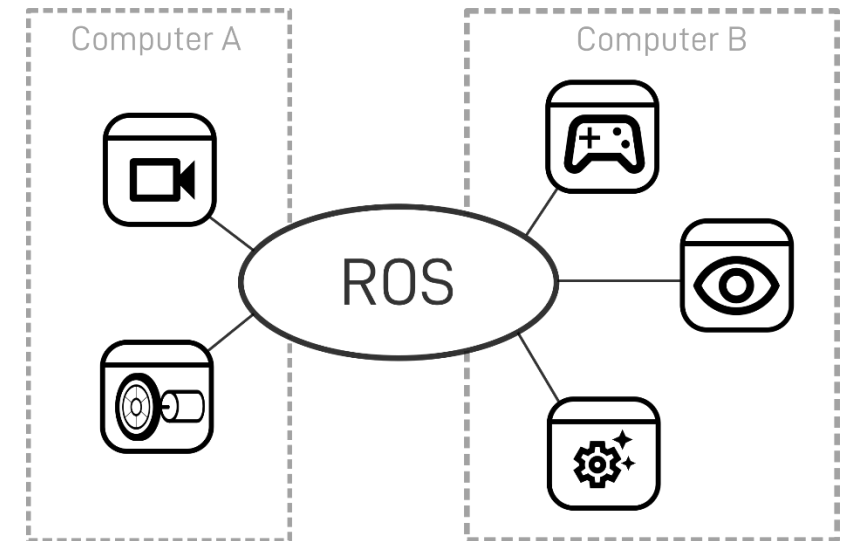


<https://tinyurl.com/488yebmz>

Nodes in a ROS 2 Network



- Multiple nodes can run **at the same time**
- Nodes on the **same machine** or **different machines** can communicate
- All active nodes form part of the **ROS 2 network**
- This flexibility allows:
 - Running everything on one computer
 - Distributing components across multiple devices



<https://tinyurl.com/488yebmz>

Working with Nodes



- Key ROS 2 commands:

```
ros2 run <package_name> <node_name>    # Run a node
ros2 node list                          # List all running nodes
```

- Example: RPLidar driver node (rplidar_node) runs on a Raspberry Pi ("robot") Another computer ("dev") can see that node over the network

Working with Nodes



RPLidar A1 node running in ROS 2

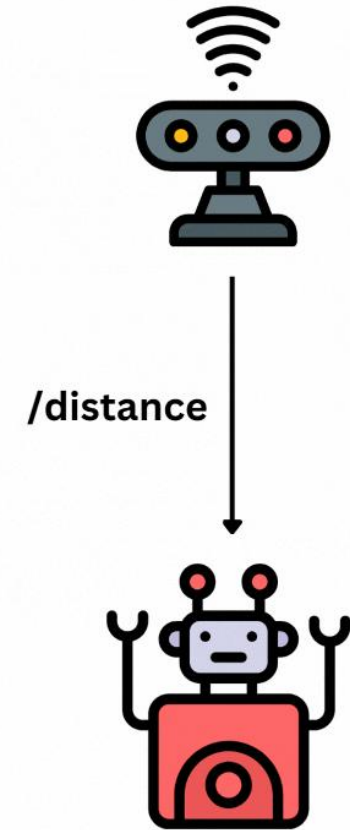
```
Pi SSH - robot@robot-pi: ~
robot@robot-pi:~$ ros2 run rplidar_ros rplidar_node
[INFO] [1627944520.747909087] [rplidar_node]: RPLIDAR running on ROS 2 package rplidar_ros. SDK Version:
1.10.0
[INFO] [1627944523.262584124] [rplidar_node]: Firmware Ver: 1.28
[INFO] [1627944523.262770136] [rplidar_node]: Hardware Rev: 5
[INFO] [1627944523.264286374] [rplidar_node]: RPlidar health status: 0
[INFO] [1627944523.807922798] [rplidar_node]: Current scan mode: Express, max_distance: 12.0 m, Point nu
mber: 4.0K , angle_compensate: 1
█

Dev Local - dev@dev-PC: ~
dev@dev-PC:~$ ros2 node list
/rplidar_node
dev@dev-PC:~$ █
```

Topics and Messages



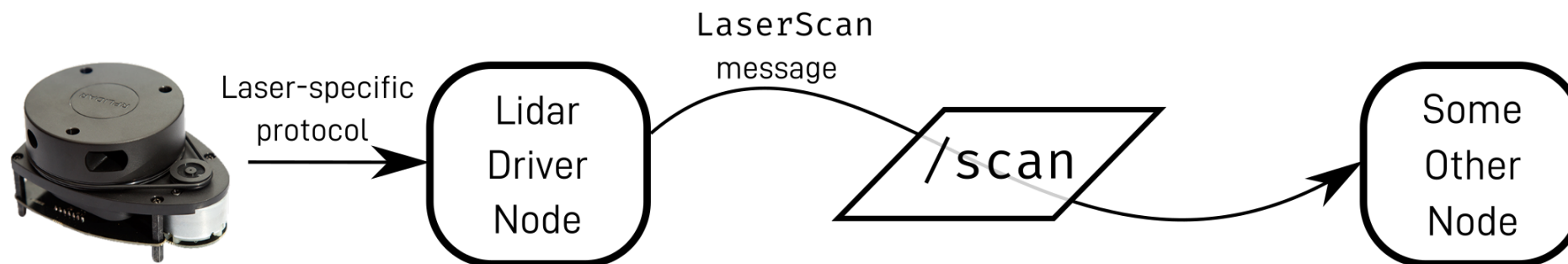
- ROS 2 nodes communicate using **topics**
- A **topic** is a named channel where nodes exchange data
- **Publishers**: send messages to a topic
- **Subscribers**: receive messages from a topic
- ROS 2 manages communication across machines — you only need topic name + message type



Why Topics?



- Allows **nodes** to work together
- Example:
 - rplidar_node **publishes LaserScan messages** to /scan
 - Other nodes subscribe to /scan **without worrying about hardware details**
- Promotes **flexibility, modularity, and reusability**



Commands for Topics



```
ros2 topic list          # See all available topics
ros2 topic echo <topic_name> # View messages being published
```

- **Example:**

```
ros2 topic echo /scan
```

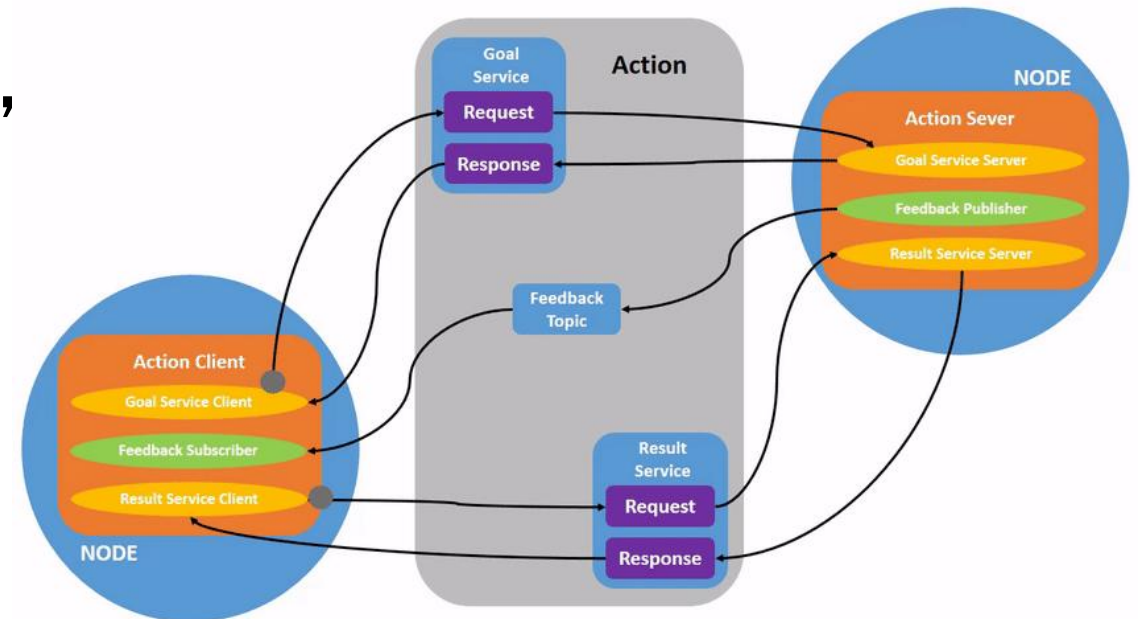
Displays laser data published by `rplidar_node`

```
Dev Local - dev@dev-PC: ~
dev@dev-PC:~$ ros2 topic list
/parameter_events
/rosout
/scan
dev@dev-PC:~$ ros2 topic echo /scan --no-arr
header:
  stamp:
    sec: 1627944846
    nanosec: 335377481
  frame_id: laser_frame
angle_min: -3.1241390705108643
angle_max: 3.1415927410125732
angle_increment: 0.01745329238474369
time_increment: 3.51982869517542e-08
scan_time: 1.2636184692382812e-05
range_min: 0.15000000596046448
range_max: 12.0
ranges: '<sequence type: float, length: 360>'
```

Services and Actions



- **Services: Synchronous communication (Request-Response model).**
 - Used for tasks like “get robot status” or “move to position.”
- **Actions: Asynchronous communication with feedback, suited for long-running tasks (e.g., “navigate to goal”).**
 - Provides goal, feedback, result mechanisms.



What are Services?

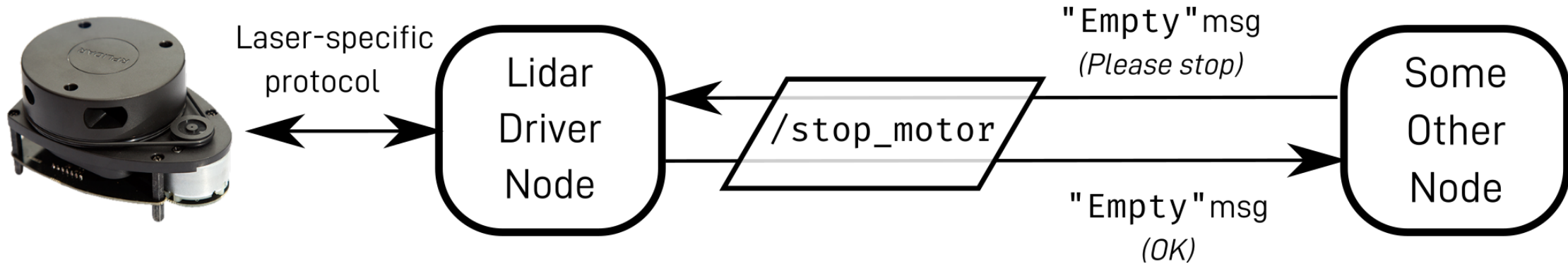


- Unlike topics (continuous streams), **services** allow one-time communication
- A node sends a **request**, another node replies with a **response**
- Useful when you want:
 - A calculation result
 - Confirmation that a request was received

Example – Lidar Motor Control



- Lidar spins continuously when active
- To save battery, we may want to **pause scanning**
- The driver provides the **/stop_motor service**
- Uses message type: **std_srvs/srv/Empty** (no extra data needed),



Commands for Services



```
ros2 service list
# Shows all available services

ros2 service call <service_name> <srv_type>
# Manually call a service
```

- Example:

```
ros2 service call /stop_motor std_srvs/srv/Empty {}
```

- Stops the lidar motor and returns an empty response

Commands for Services



- Client sends request → /stop_motor
- Server node (driver) stops the motor
- Response: **Empty message** (acknowledgment only)

```
Dev Local - dev@dev-PC: ~
dev@dev-PC:~$ ros2 service list
/rplidar_node/describe_parameters
/rplidar_node/get_parameter_types
/rplidar_node/get_parameters
/rplidar_node/list_parameters
/rplidar_node/set_parameters
/rplidar_node/set_parameters_atomically
/start_motor
/stop_motor
dev@dev-PC:~$ ros2 service call /stop_motor std_srvs/srv/Empty
waiting for service to become available...
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()
```

Node Arguments



- ROS 2 nodes can be customized at runtime using:
 - **Parameters** → change a node's behavior
 - **Remapping** → change names of topics, services, or nodes
- Enabled with **--ros-args** flag when running nodes

Node Arguments - Parameters



- **Define options a node can accept**
- **Example: Two lidar drivers, each with its own serial port**

```
ros2 run rplidar_ros rplidar_node \  
--ros-args -p serial_port:=/dev/ttyUSB0
```

- **Node uses the given serial port**

Node Arguments - Remapping



Remapping

- Allows renaming of topics, services, or nodes
- Useful when:
 - Nodes expect different topic names
 - Avoiding conflicts with multiple devices
- **Example: remap** `/scan` $\rightarrow$ `/scan_1`

```
ros2 run rplidar_ros rplidar_node \  
--ros-args -p serial_port:=/dev/ttyUSB0 -r scan:=scan_1
```

Node Arguments - Remapping



- **Example:**

```
Pi SSH - robot@robot-pi: ~
robot@robot-pi:~$ ros2 run rplidar_ros rplidar_node --ros-args -p serial_port:=/dev/ttyUSB0 -r scan:=scan_1
[INFO] [1627945217.503372270] [rplidar_node]: RPLIDAR running on ROS 2 package rplidar_ros. SDK Version: 1.10.0
[INFO] [1627945220.020525540] [rplidar_node]: Firmware Ver: 1.28
[INFO] [1627945220.020708931] [rplidar_node]: Hardware Rev: 5
[INFO] [1627945220.022187449] [rplidar_node]: RPlidar health status: 0
[INFO] [1627945220.565873185] [rplidar_node]: Current scan mode: Express, max_distance: 12.0 m, Point number: 4.0K , angle_compensate: 1
█
```

- if we check the topic list again, we can see that the node is publishing on scan_1.

```
Dev Local - dev@dev-PC: ~
dev@dev-PC:~$ ros2 topic list
/parameter_events
/rosout
/scan_1
dev@dev-PC:~$ █
```

Why Use Parameters & Remapping?

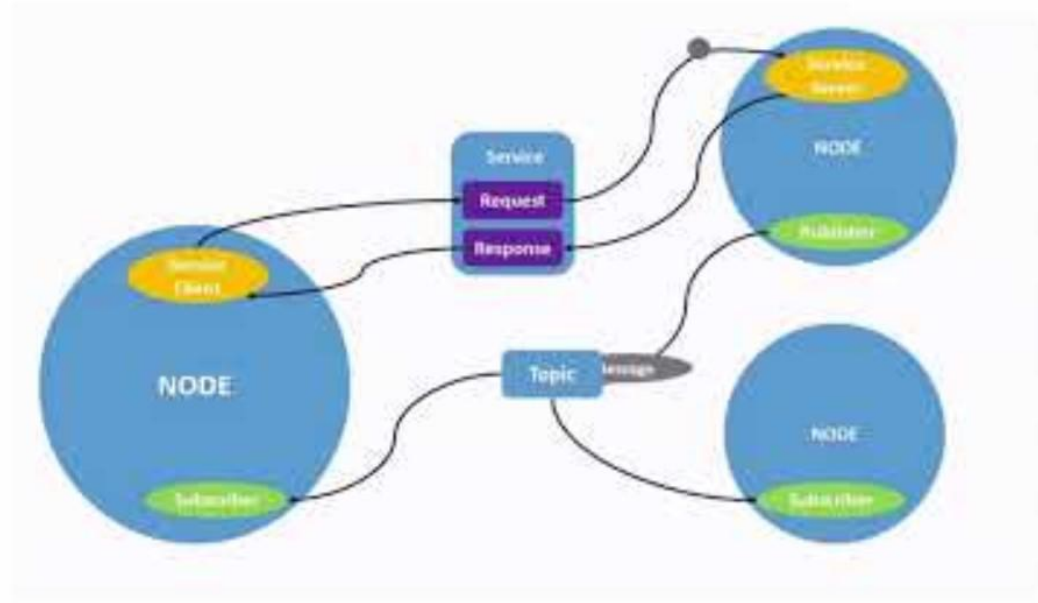


- Flexibility when running multiple instances of the same node
- Avoid name conflicts in large systems
- Cleaner and modular system design

ROS2 Architecture



Nodes

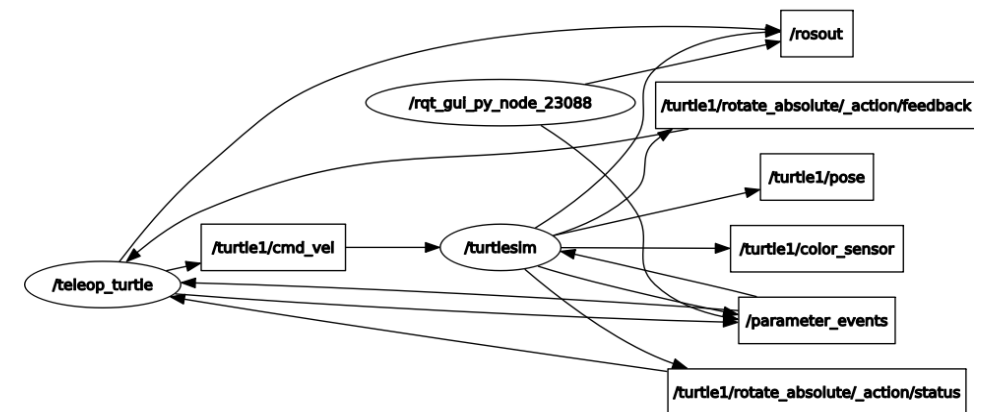
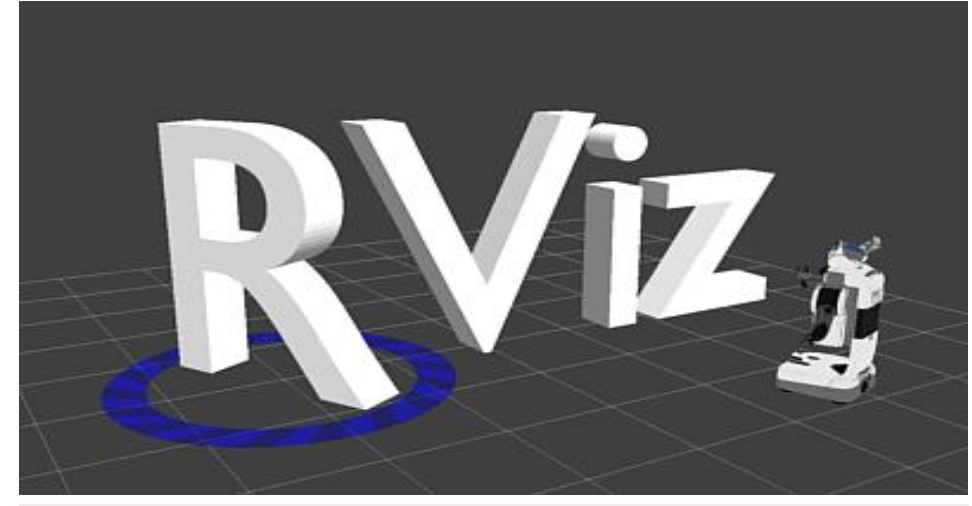


<https://www.youtube.com/watch?v=Z9R8wpQaxh8&t=117s>

Infrastructure Tools



- **ros2cli:** Command-line tools to interact with the ROS2 system.
- **rviz2:** 3D visualization tool for sensor data and robot state.
- **launch:** Launch multiple nodes with configuration files.
- **Other tools:** ros2 topic, ros2 node, ros2 service, ros graph etc.



Setting Up ROS2



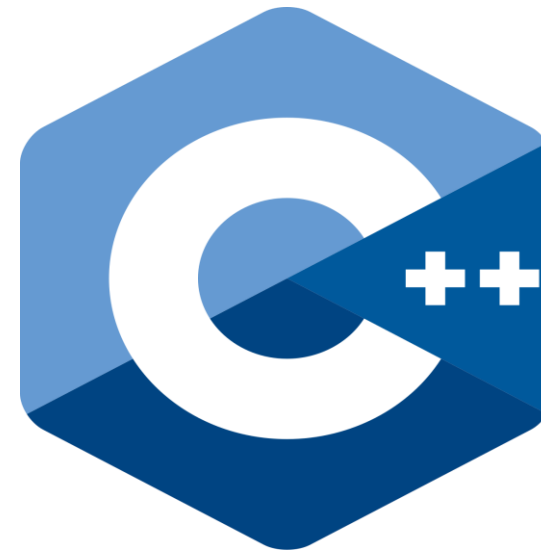
- Add system configurations and ROS2 apt repositories.
- Install ROS2 packages (ros-jazzy-desktop).
 - Follow link: <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>
- Source environment setup script to allow usage of ROS2 packages.



Developing ROS2 Nodes



- **Python (rclpy) vs C++ (rclcpp)**
 - In this course the focus will be on Python



Use Cases in IIoT



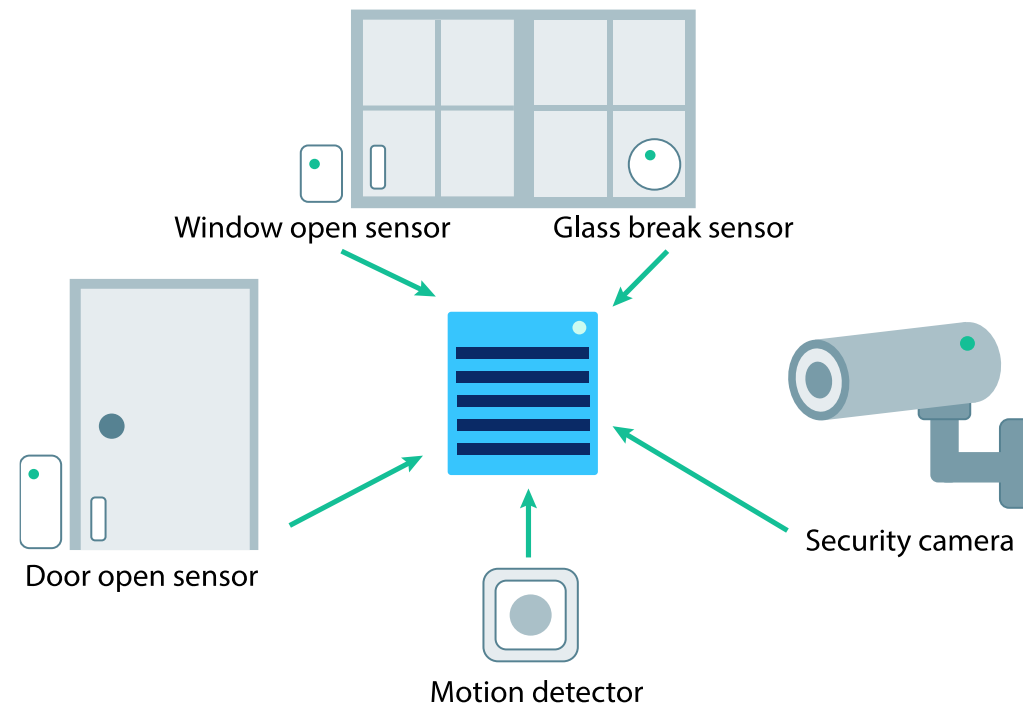
- **Robotics Systems**
 - **Pick-and-place automation**
 - **Inspection robots using ROS2 nodes for vision and control**



Use Cases in IIoT



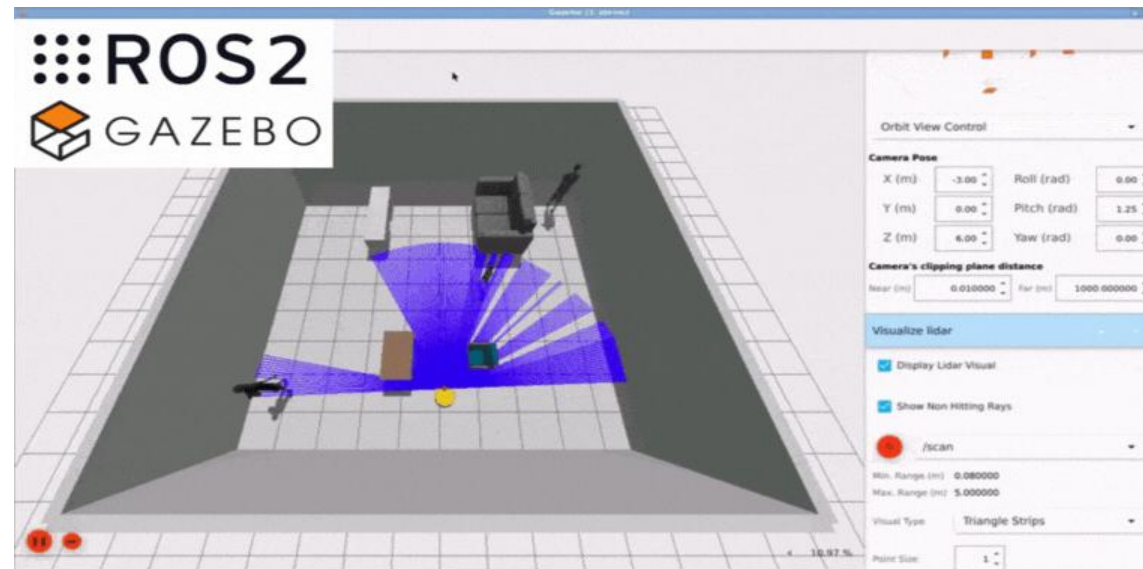
- **Sensor Networks**
 - **Distributed sensor unified communication**
 - **Nodes send real-time data to a central decision node**



Use Cases in IIoT



- **Simulations**
 - **Gazebo + ROS2 for safe testing before physical deployment**
 - **Useful for training algorithms and debugging**



Key things to remember...



Practicing outside the classroom

- *Read the following articles to better understand ROS2 and its applications*
 - <https://docs.ros.org/en/jazzy/Installation.html>
 - <https://docs.ros.org/en/jazzy/index.html>

Summary



Why ROS 2?

- **Modular, scalable, widely adopted in robotics**
- **Active community and long-term support**

Architecture

- **Nodes communicate via DDS middleware**
- **Supports distributed, multi-machine systems**

Distributions

- **Current LTS: ROS 2 Jazzy (Ubuntu 24.04)**

Core Concepts

- **Nodes → individual programs with specific tasks**
- **Topics & Messages → publish/subscribe for continuous data**
- **Services → request/response communication**
- **Parameters & Remapping → configure and customize nodes**

Key Commands

- `ros2 node list, ros2 run`
- `ros2 topic list, ros2 topic echo`
- `ros2 service list, ros2 service call`

THANK YOU.



WE ARE

HUMBER